

 BLOCK 1 – DSA: HashMap + Linked List — Session Cache

 Core Idea

- Store each user's conversation separately
 - Use HashMap → $O(1)$ lookup
 - Optional: use Linked List for eviction (LRU)
-

 Implementation

```
sessions = {}
```

```
def get_session(user_id):
```

```
    if user_id not in sessions:
```

```
        sessions[user_id] = ChatMemory()
```

```
    return sessions[user_id]
```

 LeetCode Problem

 LeetCode #146 — LRU Cache

<https://leetcode.com/problems/lru-cache/>

 Why this problem?

Sessions behave like a cache:

- users come → stored
- memory limited → old users removed

LRU Cache = HashMap + Doubly Linked List

→ $O(1)$ get and put

 Think

If 10,000 users connect:

sessions dict keeps growing ❌

How will you:

- evict old users?
 - limit memory?
-

BLOCK 2 – AI Concept: Multi-User Memory Isolation

✅ Problem

Single ChatMemory shared:

User A + User B → same context ❌

✅ Solution

Each user gets:

user_id → ChatMemory ✅

✅ Result

User A → AVL conversation ✅

User B → Heap conversation ✅

(no mixing)

✅ Insight

Memory isolation = correct AI behavior ✅

Required for real chat systems ✅

BLOCK 3 – Build: Add Sessions to FastAPI

✅ Request Model

```
class ChatRequest(BaseModel):
```

```
    user_id: str
```

```
    message: str
```

✅ Session Store

```
sessions = {}
```

```
def get_session(user_id):
```

```
    if user_id not in sessions:
```

```
        bot = ChatMemory()
```

```
        bot.prepare_document("final_clean_dsa.md")
```

```
        sessions[user_id] = bot
```

```
    return sessions[user_id]
```

✅ Endpoint

```
@app.post("/chat")
```

```
async def chat_endpoint(req: ChatRequest):
```

```
    try:
```

```
        bot = get_session(req.user_id)
```

```
        reply = bot.chat(req.message)
```

```
        return {"reply": reply}
```

```
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=str(e))
```

✓ Verify

Test with:

user_id = "A"

user_id = "B"

✓ Separate memory

✓ No context leakage

✓ Follow-up works per user

🔄 BLOCK 4 – Reflect

1. How will you limit number of sessions (like LRU)?
 2. Should sessions expire after inactivity (TTL)?
 3. Where will you store sessions in production (RAM vs Redis)?
 4. What happens if server restarts — do you lose conversations?
-

✓ ✓ SUMMARY

✓ DSA Mapping

HashMap → user → memory

LRU Cache → session eviction

✓ AI System

Multi-user chatbot ✓

Memory isolation ✓

Scalable foundation ✓
